R3.04 – Qualité de développement
Les principes SOLID

Pierre Pomeret-Coquot
pierre.pomeret@iut-rodez.fr
Université Toulouse Capitole – IUT de Rodez

2026 – 2027







Plan du cours

1. Problématique

2. Les principes SOLID
   - Principe de responsabilité unique
   - Principe ouvert/fermé
   - Principe de substitution de Liskov
   - Principe de ségrégation des interfaces
   - Principe d'inversion des dépendances

3. Résumé

CC BY

Problématique

« Mauvais » développement :

- \+ Rapide à coder
- – Difficile à maintenir
- – Difficile à faire évoluer
- – Difficile à transmettre

→ La dette technique s'accumule

« Bon » développement :

- – Long à coder
- \+ Facile à maintenir
- \+ Facile à faire évoluer
- \+ Facile à transmettre

→ Pas/peu de dette technique

La dette technique rend les modifications de plus en plus difficiles.

CC BY

Assurer un développement de qualité

Plusieurs approches complémentaires :

- Allouer du temps et des moyens
- S'inspirer des bonnes pratiques éprouvées
  - Suivre les principes SOLID
  - Utiliser des patrons de conception adaptés
  - etc.
- Utiliser des outils adaptés
- Documenter le code et la travail effectué
- etc.

CC BY

Les principes SOLID

Acronyme lié aux 5 principes clés pour obtenir un logiciel qualitatif :

- Single responsability principle
- Open/closed principle
- Liskov's substitution principle
- Interface segregation principle
- Dependency inversion principle

→ Faible couplage et forte cohésion

- Faible **couplage** = les modules sont peu dépendants les uns des autres
- Forte **cohésion** = les tâches réalisées par un seul module sont fonctionnellement reliées

→ Évolutivité du logiciel

→ Réduction du risque de bugs liés à l'architecture

→ etc.

CC BY

Le principe de responsabilité unique

SOLID

Single responsibility principle

Le déroulement d'une fonctionnalité est permis par plusieurs classes en interaction.

Chaque classe possède la responsabilité d'une partie du traitement.

Une classe = une responsabilité

⚠ Responsabilités multiples → couplage → fragilité.

CC BY

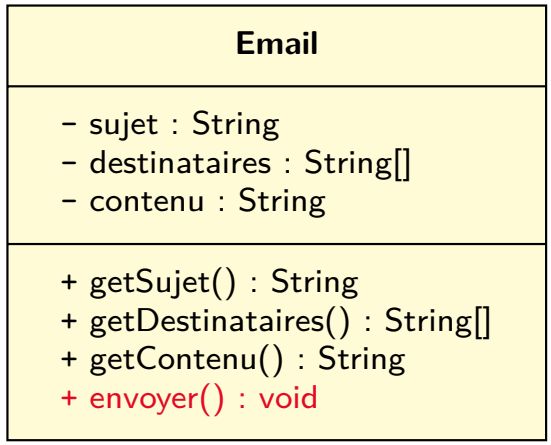
Exemple

```java
class Email {
   private String sujet;
   private String[] destinataires;
   private String contenu;

   public Email(String sujet, String[] destinataires, String contenu) {
      this.sujet = sujet;
      this.destinataires = destinataires;
      this.contenu = contenu;
   }

   public String getSujet() {return sujet;}
   public String[] getDestinataires() {return destinataires;}
   public String getContenu() {return contenu;}

   public void envoyer() {
      //Code complexe pour envoyer un mail...
   }
}
```




```java
class Main {
  public static void main(String[] args) {
    Email m = new Email("Hello", new String[]{"test@example.com"}, "Hello world!");
    m.envoyer();
  }
}
```

CC BY

Solution → plusieurs classes

```java
class Email {
    private String sujet;
    private String[] destinataires;
    private String contenu;

    public Email(String sujet, String[] destinataires, String contenu) {
        this.sujet = sujet;
        this.destinataires = destinataires;
        this.contenu = contenu;
    }

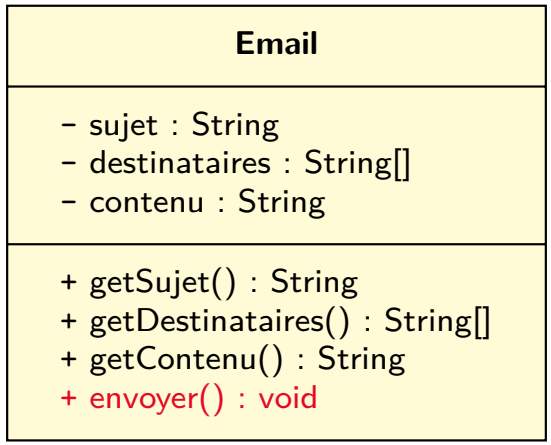
    public String getSujet() {return sujet;}
    public String[] getDestinataires() {return destinataires;}
    public String getContenu() {return contenu;}
}
```

```java
class ServeurMail {
    public void envoyerMail(Email mail) {
        //Code complexe pour envoyer un mail...
    }
}
```

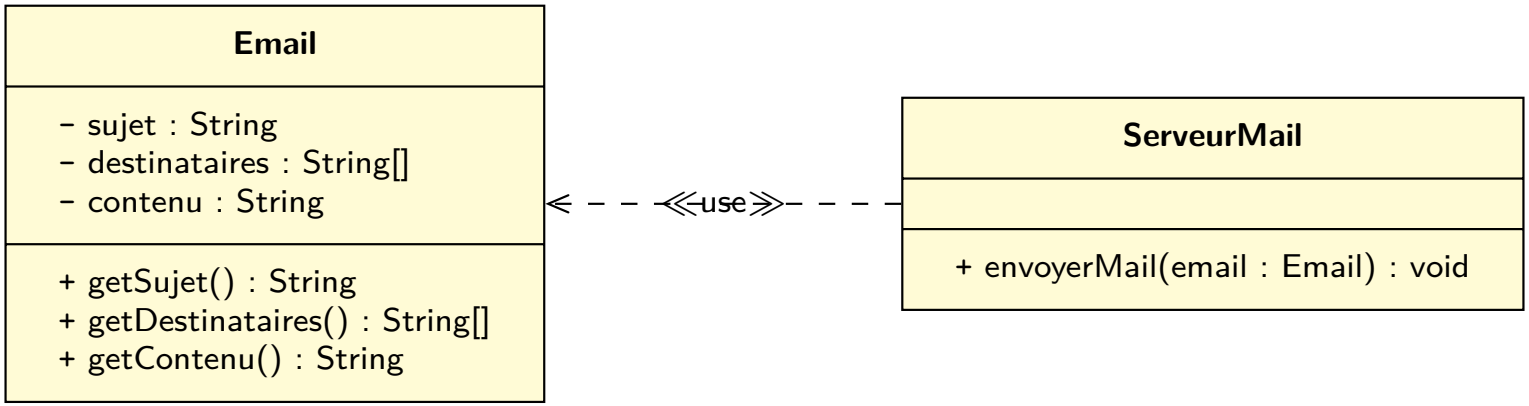
```java
class Main {
    public static void main(String[] args) {
        Email e = new Email("Hello", new String[]{"test@example.com"}, "Hello world!");
        ServeurMail serveur = new ServeurMail();
        serveur.envoyerMail(e);
    }
}
```

CC BY

Solution → plusieurs classes



devient



CC BY

Le principe ouvert/fermé

SOLID

Open-closed principle

- Ouvert aux extensions
- Fermé aux modifications

→ Possible d'étendre les fonctionnalités de comportement sans avoir besoin de modifier le code source !

CC BY

Exemple

```java
class FigureGeometrique {

    private String typeFigure;

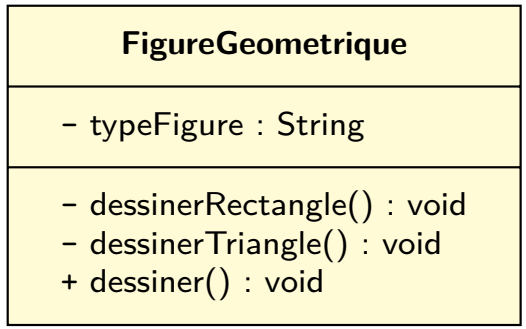
    public FigureGeometrique(String typeFigure) {
        this.typeFigure = typeFigure;
    }

    public void dessiner() {
        if(typeFigure.equals("rectangle")) {
            dessinerRectangle();
        }
        else if(typeFigure.equals("triangle")) {
            dessinerTriangle();
        }
    }

    private void dessinerRectangle() {
        //Code pour dessiner un rectangle...
    }

    private void dessinerTriangle() {
        //Code pour dessiner un triangle...
    }

}
```



CC BY

Solution → héritage

```java
abstract class FigureGeometrique {

  public abstract void dessiner();

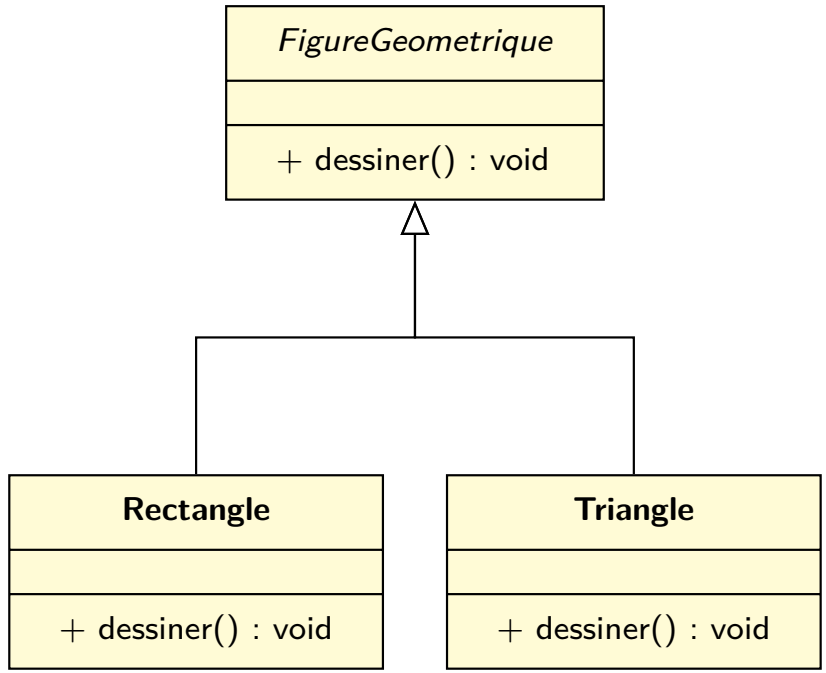
}

class Rectangle extends FigureGeometrique {

  public void dessiner() {
    //Code pour dessiner un rectangle...
  }
}

class Triangle extends FigureGeometrique {

  public void dessiner() {
    //Code pour dessiner un triangle...
  }
}
```



CC BY

Le principe de substitution de Liskov

SOLID

Liskov's substitution principle

- Un objet d'une super-classe doit pouvoir être remplacé par n'importe quel objet de ses sous-classes
- Idée de *programmation par contrat* : les classes filles respectent les mêmes spécifications que leurs classes-mères.

→ Ne pas utiliser l'héritage si le contrat n'est pas respecté (mais, p.ex. la composition)

CC BY

Exemple

```java
class Rectangle {

   private double longueur;
   private double largeur;

   public Rectangle(double longueur, double largeur) {
      this.longueur = longueur;
      this.largeur = largeur
   }

   public void setLongueur(double longueur) { this.longueur = longueur; }
   public void setLargeur(double largeur)   { this.largeur = largeur;   }

   public double getLongueur() { return longueur; }
   public double getLargeur()  { return largeur;  }

   public double calculerAire() { return longueur*largeur; }
}
```

```java
class Carre extends Rectangle {

   public Carre(double cote) {
      super(cote, cote);
   }
}
```

Est-ce que cela vous convient ?

CC BY

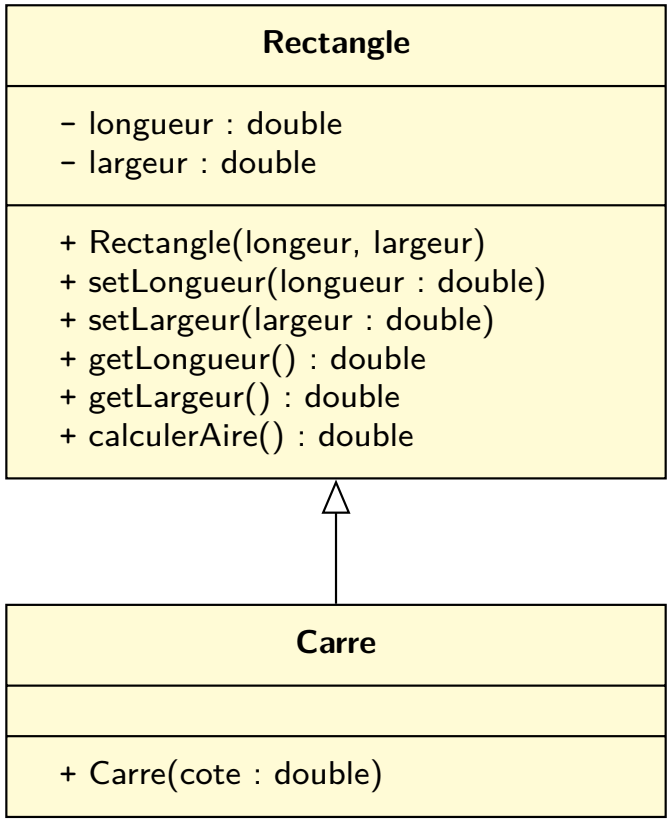
Exemple

- Le **contrat** de `Rectangle` implique de pouvoir changer la longueur et la largeur indépendamment

```java
@Test
public void testerRectangle() {
    Rectangle r = new Rectangle(4, 4);
    r.setLargeur(10);
    assertEquals(10, r.getLargeur());
    assertEquals(4, r.getLongueur());
    assertEquals(40, r.calculerAire());
}
```

- Ce n'est pas le contrat de `Carre`.

```java
@Test
public void testerCarre() {
    Carre c = new Carre(4);
    c.setLargeur(10);
    assertEquals(10, c.getLargeur());
    assertEquals(10, c.getLongueur());
    assertEquals(100, c.calculerAire());
}
```



CC BY

Exemple

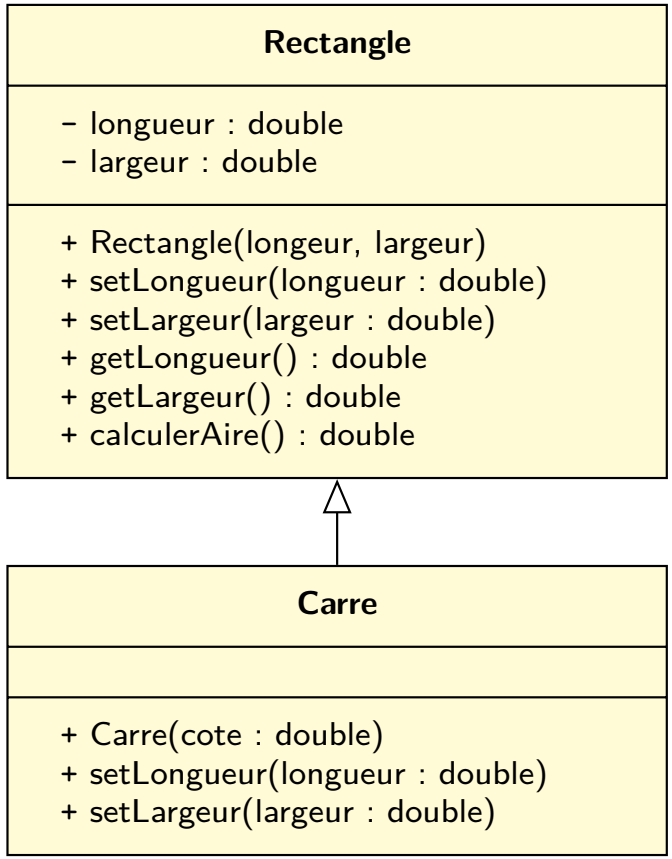
```java
class Carre extends Rectangle {

    public Carre(double cote) {
        super(cote, cote);
    }

    @Override
    public void setLongueur(double longueur) {
        super.setLongueur(longueur);
        super.setLargeur(longueur);
    }

    @Override
    public void setLargeur(double largeur) {
        super.setLongueur(longueur);
        super.setLargeur(longueur);
    }
}
```

```java
@Test
public void testerRectangleAvecUnCarre() {
    Rectangle r = new Carre(4);
    r.setLargeur(10);
    assertEquals(10, r.getLargeur());
    assertEquals(4, r.getLongueur());
    assertEquals(40, r.calculerAire());
}
```



CC BY

Exemple

Et des erreurs en prévision !

```java
public void redimensionner(Rectangle rect, double factor) {
    rect.setLongueur(rect.getLongueur() * factor);
    rect.setLargeur(rect.getLargeur() * factor);
}
```

```java
@Test
public void testerRedimensionnerRectangle() {
    Rectangle r = new Rectangle(4, 4);
    redimensionner(r, 2);
    assertEquals(8, r.getLargeur());
    assertEquals(8, r.getLongueur());
    assertEquals(64, r.calculerAire());
}
```

```java
@Test
public void testerRedimensionnerCarre() {
    Carre c = new Carre(4);
    redimensionner(c, 2);
    assertEquals(8, r.getLargeur());
    assertEquals(8, r.getLongueur());
    assertEquals(64, r.calculerAire());
}
```

CC BY

Solution → composition

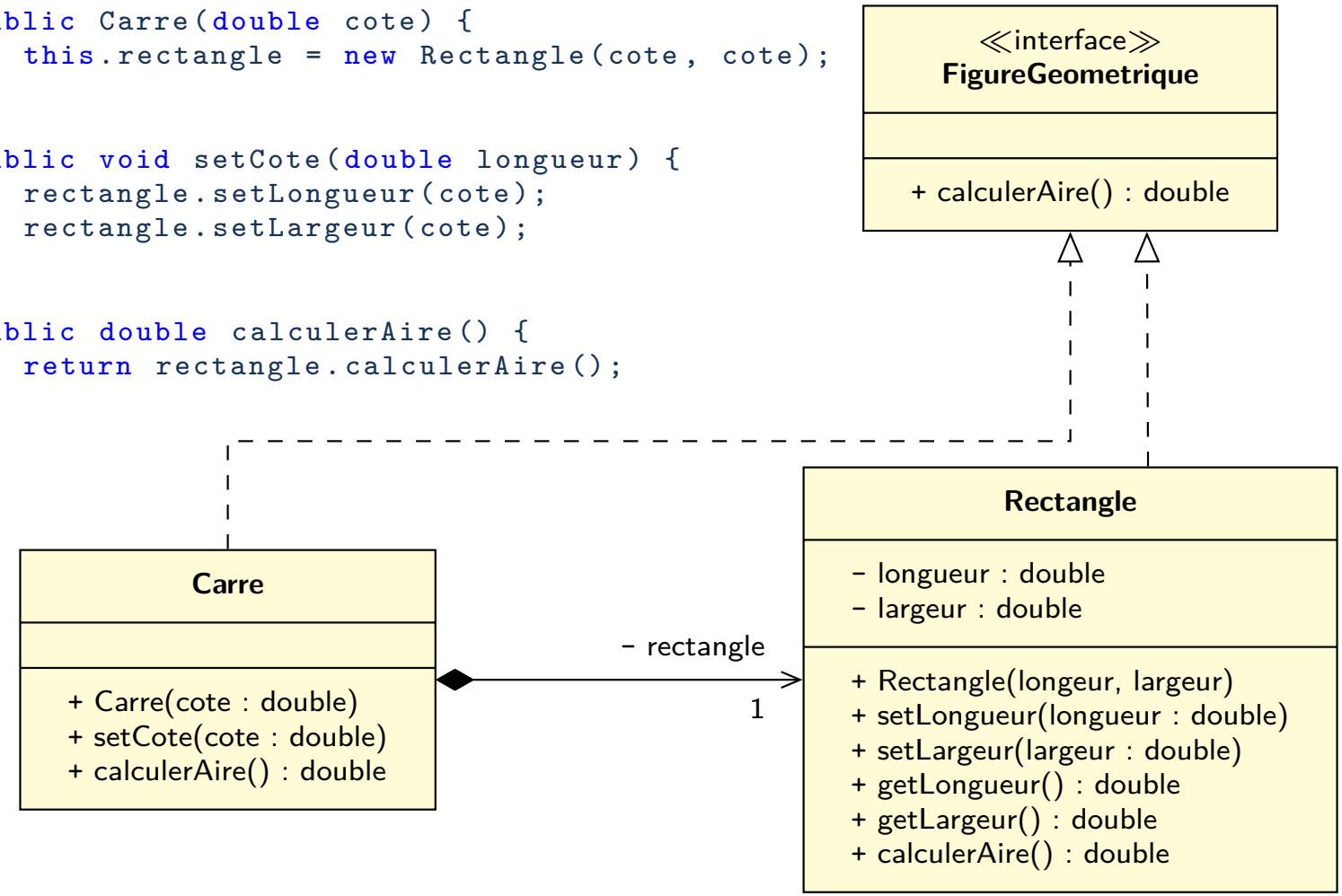
```java
class Carre implements FigureGeometrique {

    private Rectangle rectangle;

    public Carre(double cote) {
        this.rectangle = new Rectangle(cote, cote);
    }

    public void setCote(double longueur) {
        rectangle.setLongueur(cote);
        rectangle.setLargeur(cote);
    }

    public double calculerAire() {
        return rectangle.calculerAire();
    }
}
```



CC BY

Le principe de ségrégation des interfaces

SOLID

Interface segregation principle

- Un objet **ne doit pas** dépendre de méthodes qu'il n'utilise pas !
- Donc, une interface ne doit pas définir dans son contrat des méthodes qui ne pourront pas être implémentées.

CC BY

Exemple

```
1  interface I {
2     void opGlobale() ;
3     void opA() ;
4     void opB() ;
5  }
```

```
1  class X implements I {
2     public void opGlobale() {
3        /* ici le code */
4     }
5     void opA() {
6        /* ici le code */
7     }
8     void opB() {
9        /* ici le code */
10    }
11 }
```

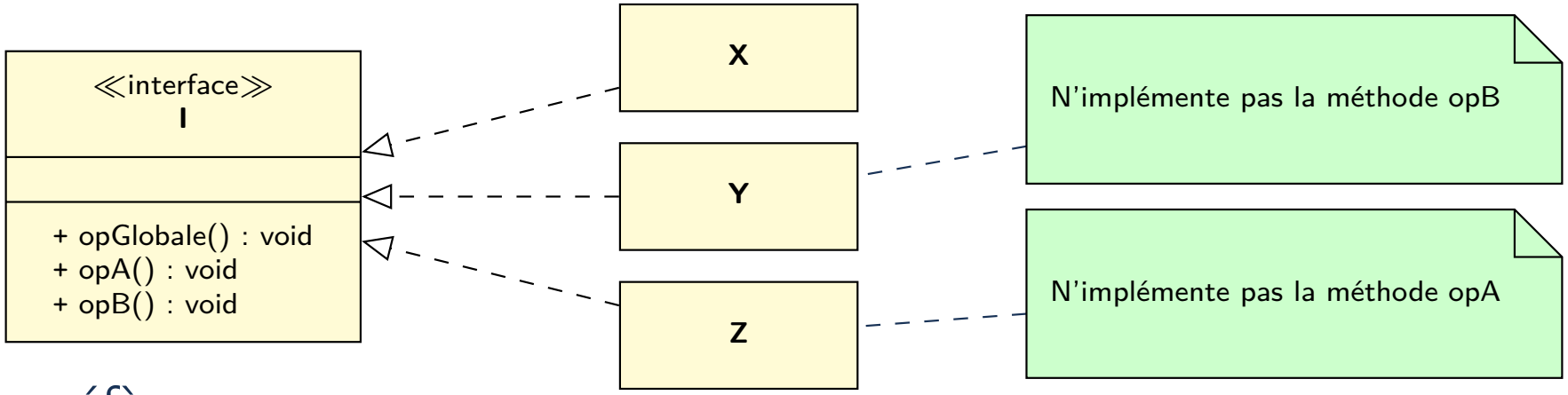
```
1  class Y implements I {
2     public void opGlobale() {
3        /* ici le code */
4     }
5     void opA() {
6        /* ici le code */
7     }
8     void opB() {
9        throw new Error("Impossible à implémenter");
10    }
11 }
```

```
1  class Z implements I {
2     public void opGlobale() {
3        /* ici le code */
4     }
5     void opA() {
6        throw new Error("Impossible à implémenter");
7     }
8     void opB() {
9        /* ici le code */
10    }
11 }
```

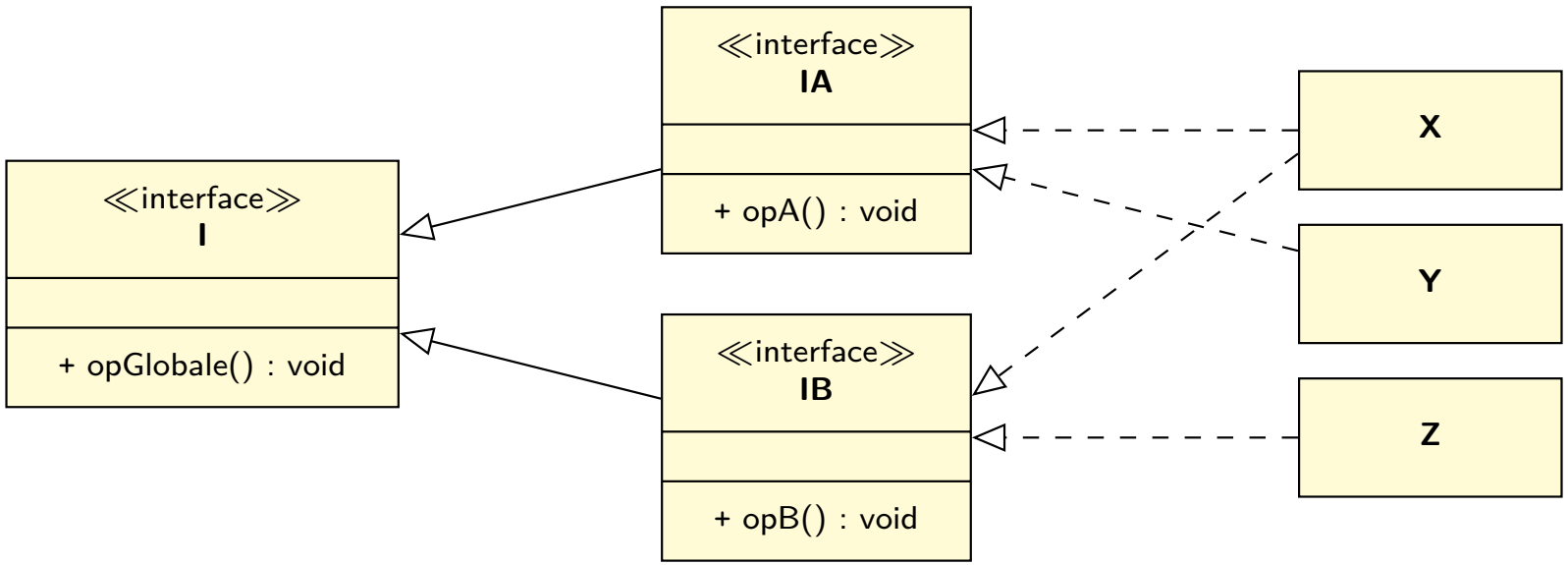
CC BY

Solution → interfaces en cascade

Le modèle initial :



On préfèrera :



CC BY

Le principe d'inversion des dépendances

SOLID

Dependency inversion principle

- Les classes ne doivent pas dépendre d'implémentations concrètes, mais d'abstractions (classes abstraites, interfaces)
- Les abstractions de doivent pas dépendre des implémentations.

CC BY

Exemple

```java
public class Image {

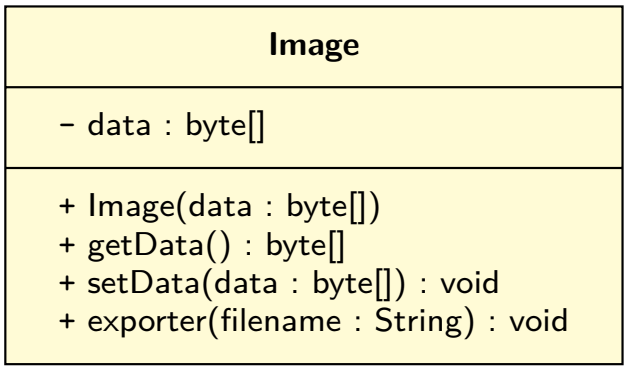
    private byte[] data

    public Image(byte[] data) {
        this.data = data;
    }

    public byte[] getData() {
        return data;
    }

    public void setData(byte[] data) {
        this.data = data;
    }

    public void exporter(String filename) {
        /* Traitement pour l'export en JPEG */
    }
}
```



Problème : ne respecte pas le principe de responsabilité unique !

CC BY

Exemple

```java
public class ExportJPG {
    public void exporter(byte[] d, String f) {
        /* Traitement pour l'export en JPEG */
    }
}
```

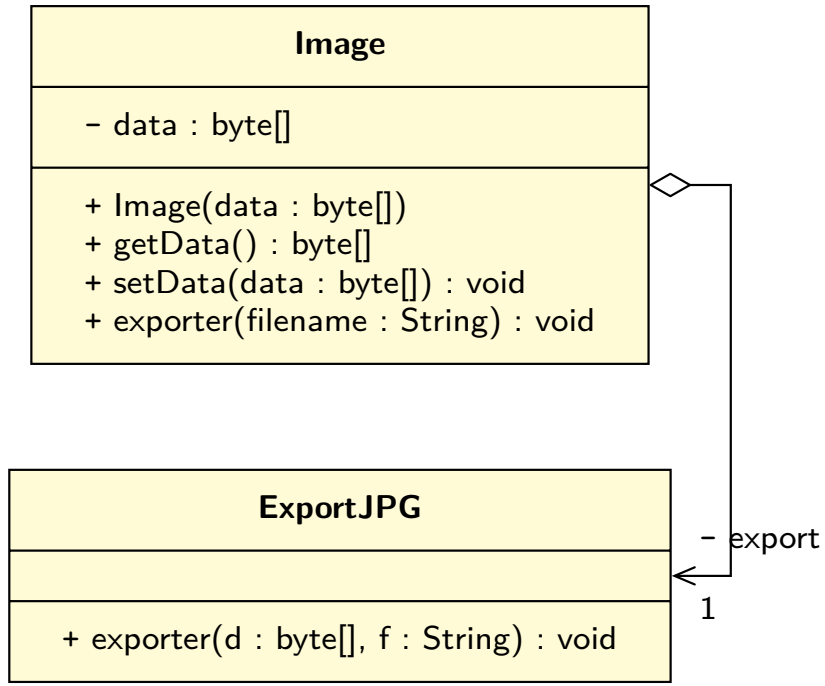
```java
public class Image {

    private byte[] data
    private ExportJPG export;

    public Image(byte[] data) {
        this.data = data;
        this.export = new ExportJPG();
    }

    /* Getters et setters */

    public void exporter(String filename) {
        export.exporter(data, filename);
    }
}
```



Et si on veut exporter en PNG ? en TIFF ? etc.

Le problème est que `Image` dépend d'une implémentation concrète !

CC BY

Solution → dépendance envers une abstraction

```java
interface Export {
    public void exporter(byte[] d, String f);
}
```

```java
public class ExportJPG implements Export {
    public void exporter(byte[] d, String f) {
        /* Traitement pour l'export en JPEG */
    }
}
```

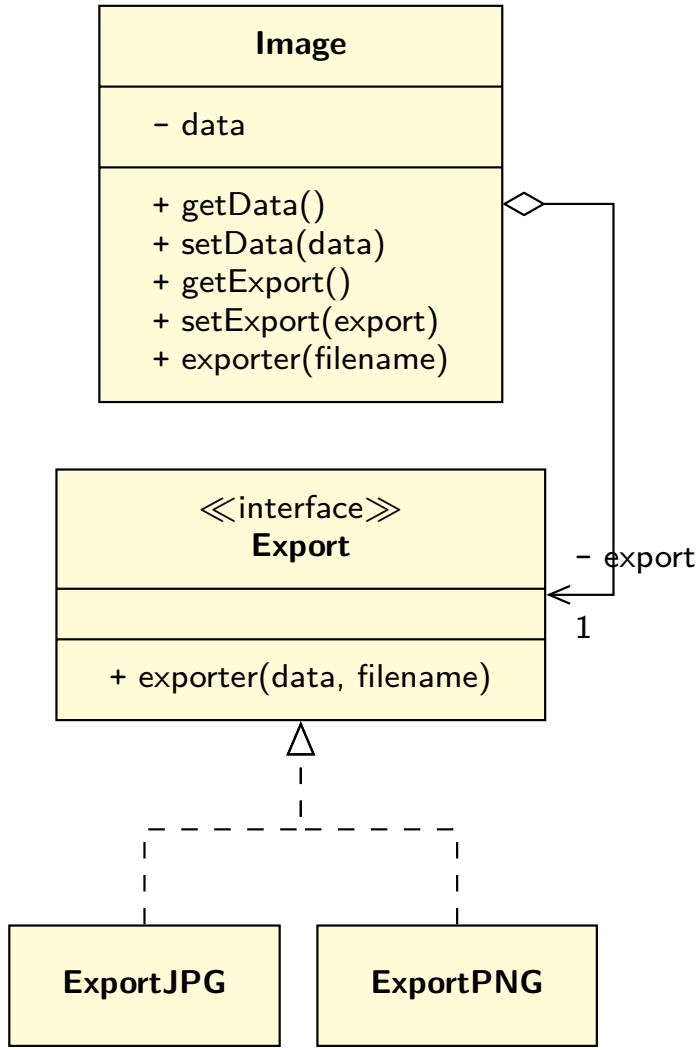
```java
public class Image {

    private byte[] data
    private Export export;

    public Image(byte[] data, Export export) {
        this.data = data;
        this.export = export;
    }

    /* Getters et setters */

    public void exporter(String filename) {
        export.exporter(data, filename);
    }
}
```



CC BY

Résumé (très, très, très simplifié)

- Single responsability principle
  - Ne pas mettre tout le code dans une classe...
  - \+ Mais diviser en plusieurs classes dédiées.
- Open/closed principle
  - Ne pas gérer la variété des comportements avec des if...then...
  - \+ Mais utiliser l'héritage.
- Liskov's substitution principle
  - Ne pas utiliser l'héritage lorsque les contrats sont différents...
  - \+ Mais utiliser la composition.
- Interface segregation principle
  - Ne pas inclure dans l'interface des méthodes non-implémentables...
  - \+ Mais diviser en plusieurs interfaces en cascade.
- Dependency inversion principle
  - Ne pas faire dépendre de classes concrètes...
  - \+ Mais faire dépendre de classes abstraites et interfaces.

CC BY